Sample Package Creation and Git Version Control

Free Open Source Software | Assignment 2

Submitted by
Yuvaraj R
24BCS187

Aim

To create a sample software package, host it in a Git repository, check out a tracked file, modify it, and push the updated version to the remote repository.

Work completed

I created a Python package named foss-geometry-toolkit for 2D geometric calculations and hosted it in my College repository on GitHub. I first staged, committed, and pushed an initial version providing rectangle area and perimeter calculations (version 0.1.0). I then checked out the calculation source file from HEAD, implemented circle area and circumference computations, updated the public module exports and example script, bumped the package version to 0.2.0, and committed and pushed the updated package.

Tools and repository

Tools used: Python 3.9 or later, Git, GitHub, and a terminal. Git records the version history; GitHub hosts the remote repository.
Repository: https://github.com/<your-username>/<your-repo-name>
Branch: assignment-2-geometry-package
Package version: 0.1.0 initially, updated to 0.2.0
Open the assignment branch on GitHub

Package structure

```
Assignment-2/
  pyproject.toml
  README.md
  .gitignore
  example.py
  geometry_toolkit/
    __init__.py
    calculator.py
  tests/
    test_calculator.py
```

pyproject.toml defines the package metadata and build configuration under PEP 621. calculator.py contains the geometric calculation functions, __init__.py exposes them for import, example.py demonstrates their use, and test_calculator.py verifies accuracy. The package has no runtime dependencies.

Procedure followed

1. I fetched the latest main branch and created a separate worktree and assignment branch. This kept unrelated local changes in the College repository intact.

```
git fetch origin main
git worktree add -b assignment-2-geometry-package \
```

```
  /tmp/foss-assignment-2-worktree origin/main
cd /tmp/foss-assignment-2-worktree
```

The commands below use P as a short name for the assignment directory. This is equivalent to using the full path in each command.

```
P="Semester 5/Free Open Source Software/Assignment-2"
```

2. I created the package files with the initial rectangle area and perimeter functions, staged them, committed the package, and pushed the branch to GitHub.

```
git add -- "$P"
git commit -m "Add sample Python geometry calculation package"
git push -u origin assignment-2-geometry-package
```

Initial commit: [initial-commit-hash] (e.g. 313a073). The first push created the remote assignment branch on GitHub.

3. I checked out the tracked calculation source file from HEAD before editing it.

```
git checkout HEAD -- "$P/geometry_toolkit/calculator.py"
```

HEAD refers to the current commit. This command restores the selected file from that commit; it does not lock the file. I performed the checkout before making the new changes.

4. I added circle_area and circle_perimeter to calculator.py, exported them through __init__.py, updated example.py, and changed the package version to 0.2.0. I also updated the README with the workflow.

5. I checked the changes, committed the update, and pushed the branch again.

```
git diff --check
git add -- "$P"
git commit -m \
  "Add circle geometry computations after file checkout"
git push origin assignment-2-geometry-package
```

Updated commit: [updated-commit-hash] (e.g. 4f41ddf). The second push advanced the remote branch from [initial-commit-hash] to [updated-commit-hash].

Commit evidence

```
[initial-commit-hash]  Add sample Python geometry calculation package
[updated-commit-hash]  Add circle geometry computations after file checkout
```

(Replace [initial-commit-hash] and [updated-commit-hash] with the actual commit hashes produced by git log --oneline -2 after performing the steps.)

Source code change

The initial calculator.py contained rectangle_area and rectangle_perimeter. After the checkout, I added circle_area and circle_perimeter. The final source file is shown below.

```
"""2D geometric dimension and area computation functions."""

import math


def rectangle_area(length: float, width: float) -> float:
    """Calculate and return the area of a rectangle."""
    if length < 0 or width < 0:
        raise ValueError("Dimensions must be non-negative.")
    return float(length * width)

def rectangle_perimeter(length: float, width: float) -> float:
    """Calculate and return the perimeter of a rectangle."""
```

```
    if length < 0 or width < 0:
        raise ValueError("Dimensions must be non-negative.")
    return float(2 * (length + width))


def circle_area(radius: float) -> float:
    """Calculate and return the area of a circle."""
    if radius < 0:
        raise ValueError("Radius must be non-negative.")
    return float(math.pi * (radius ** 2))


def circle_perimeter(radius: float) -> float:
    """Calculate and return the circumference (perimeter) of a circle."""
    if radius < 0:
        raise ValueError("Radius must be non-negative.")
    return float(2 * math.pi * radius)
```

Execution and output

I ran the example from the Assignment-2 directory using the following command.

```
python3 example.py
```

Observed / Expected output

```
Rectangle (Length: 5.0, Width: 3.0): Area = 15.0, Perimeter = 16.0
Circle (Radius: 7.0): Area = 153.94, Circumference = 43.98
```

Verification

I checked both geometric functions using unit squares, standard rectangles, zero dimensions, and standard and fractional circle radii. Floating-point comparisons used standard precision tolerance. All test cases passed.

Test case	Input Dimensions	Calculated Area	Calculated Perimeter
Unit square	Length = 1, Width = 1	1.0	4.0
Standard rectangle	Length = 5, Width = 3	15.0	16.0
Zero dimension	Length = 0, Width = 10	0.0	20.0
Unit circle	Radius = 1.0	3.1416	6.2832
Standard circle	Radius = 7.0	153.9380	43.9823

git diff --check reported no whitespace errors. The remote branch was verified at commit [updated-commit-hash] after the second push.

Result

The sample package was created, committed, and hosted on GitHub. A tracked source file was checked out and modified, and the updated package was pushed successfully. The two commits record the initial package and the subsequent enhancement.